一个新的
视角
为更智能的
计算
未来

更互联。
更自适应。
更高效。

协同操作系统

协同操作系统

综合文档

协同操作系统 (SOS)

一种基于拓扑的、自适应的、资源高效的
操作系统，灵感来源于巴克敏斯特·富勒的协同学。

整体是
不仅大于总和

其部分，
但大于
各部分的
孤立状态。

- 巴克敏斯特·富勒

灵感来源于
巴克敏斯特·富勒。
为未来而建。



拓扑
模型关系
在过程
和资源之间。

自适应调度
响应变化的
工作负载和
系统条件。

资源效率
以更少的资源
做更多的事情。

新兴协调
地方决策。
系统范围的利益。

作者
所罗门·雅威·阿德克洛
创始人兼发起人

官方网站
https://synergetic-os.github.io/

GitHub 组织
https://github.com/synergetic-os

协同操作系统 (SOS)

愿景、原则、架构和研究的综合指南

作者： Solomon Yaw Adeklo
协同操作系统的创始人和发起人

官方网站：https://synergetic-os.github.io/
GitHub 组织：https://github.com/synergetic-os
创始人简介：https://www.f6s.com/solomon-yaw-adeklo

文档：
https://www.researchgate.net/publication/408461895_A_comprehensive_systems_documentation_of_Synergetics_operating_system

对一种新计算方法的探索，该方法受到巴克敏斯特·富勒的协同学、系统思维以及追求以更少的资源做更多事情的启发。

前言：为什么这项研究很重要

计算机已成为现代生活中不可或缺的工具。它们帮助我们沟通、管理业务、进行科学研究、设计建筑、教育学生，以及开发曾经难以想象的技术。

然而，在每个计算机应用程序背后，还有一块大多数人很少考虑的软件：操作系统。

操作系统决定了计算机如何运行程序、分配内存、与硬件通信、管理文件，以及协调机器内部发生的众多活动。

几十年来，操作系统通过硬件、软件工程、调度算法、安全性和资源管理的进步不断演变。这些发展产生了强大而可靠的计算平台。

然而，计算的持续演变也创造了调查系统架构替代方法的机会。

进程与资源之间的关系能否被更明确地建模？
计算机能否更智能地适应变化的工作负载进行资源分配？

几何、拓扑、系统科学和自适应协调的理念能否激发新的计算组织方式?

这些问题为协同操作系统（SOS）提供了起点。

SOS是一个实验性的开源研究项目，探索一种以拓扑为导向和关系感知的方法来设计操作系统。它从巴克敏斯特·富勒的协同学（Synergetics）及其通过更好的设计以更少的资源实现更多目标的广泛兴趣中汲取智力灵感。

本文件解释了SOS的动机、智力基础、提议架构、研究方法、开发状态以及长期可能性，以便技术和非技术背景的读者都能理解。

本文件中有一个重要的区别：SOS是一个研究方向，而不是声称已经构建了一个完全可操作的替代现有操作系统的解决方案。其提议的优势必须通过实施、测试、测量和独立的技术评审来证明。

第一章：理解操作系统

1.1 什么是操作系统?

想象一个大型餐厅。

餐厅有厨师、服务员、顾客、食材、烹饪设备、桌子和有限的厨房空间。所有这些元素必须协同工作。如果每个厨师同时尝试使用同一个烤箱，就会造成混乱。如果在需要时食材不可用，订单就会延迟。如果没有人协调工作，餐厅就会变得低效。

餐厅经理帮助协调这些活动，决定优先事项、可用资源和竞争需求。

操作系统在计算机内部执行类似的协调角色。

计算机有处理器、内存、存储和其他硬件。应用程序请求访问这些资源，通常是同时进行的。操作系统帮助协调它们的使用。

例如，当某人在浏览互联网和下载文件时听音乐，操作系统会帮助这些活动共享计算机的资源。

1.2 操作系统的主要职责

组件	简单解释
进程管理	组织当前正在运行的程序。
CPU 调度	确定哪个程序或任务获得处理器时间。
内存管理	分配和保护计算机的工作内存。
文件管理	组织信息的存储和检索方式。
设备管理	协调与硬件（如键盘、屏幕和网络设备）的通信。
安全性和权限	帮助控制哪些程序和用户可以访问特定资源。
用户界面	提供人们与计算机互动的方式。

这些责任是相互关联的。一个程序可能需要处理器时间、内存、存储访问和网络通信来完成一个任务。

操作系统必须协调这些请求，同时保持响应性、可靠性、安全性和公平性。

1.3 为什么操作系统设计仍然重要

计算机在工作负载和限制方面差异很大。手机、科学工作站、云服务器、嵌入式控制器和超级计算机面临的资源管理问题并不相同。

即使在一台计算机上，需求也在不断变化。刚刚不重要的任务可能会变得紧急。程序可能会突然请求额外的内存或开始进行密集计算。

因此，操作系统研究人员继续探索改进调度、资源利用、安全性、能耗和系统响应能力的方法。

SOS进入这个既定领域，提出了一个特定的研究问题：

明确建模工作负载、资源和系统组件之间的关系，是否能支持自适应计算的新方法？

这个问题并不假设现有操作系统普遍低效。相反，它询问是否可以通过另一种架构视角为特定类别的工作负载提供有用的想法。

第2章：介绍协同操作系统

2.1 什么是SOS?

协同操作系统是一个实验性研究计划，探索与协同学相关的概念如何影响计算机过程和资源的组织与协调。

其提议的方法强调关系、基于拓扑的表示、资源感知决策、自适应调度和反馈。

用通俗的语言来说，SOS研究理解计算机系统各部分之间的关系是否能帮助我们更好地决策这些部分如何协同工作。

考虑一个简单的例子。

一台计算机正在运行三项活动：

- 一个需要持续响应的视频会议。
- 一个可以容忍延迟的后台文件传输。
- 一个使用大量处理器时间的数据处理任务。

资源管理系统必须决定如何平衡它们之间的竞争需求。它可能会考虑优先级、资源可用性、任务之间的依赖关系，以及延迟某项活动的后果。

SOS 探索是否更丰富的关系表示可以支持更明智的调度决策。

这是一项研究假设。它是否能产生可测量的好处取决于
所评估的模型、算法、实现和工作负载。

2.2 “协同” 的含义

“协同（synergetic）”一词在这里指的是对相互作用的组件如何影响更大系统行为的兴趣。

一辆自行车提供了一个简单的例子。它的轮子、车架、踏板、链条和刹车具有不同的独立功能。自行车的整体行为取决于这些组件如何连接和协同工作。

改变一个连接或组件可能会影响整辆自行车的行为。

计算机表现出类似的关系。内存分配的变化可能会影响处理器的活动。处理器调度的决策可能会影响应用程序的响应能力。网络操作可能会触发额外的计算。

SOS研究如何在系统设计中表示和推理这些关系。

2.3 主要研究方向

该项目汇集了几个相关的想法：

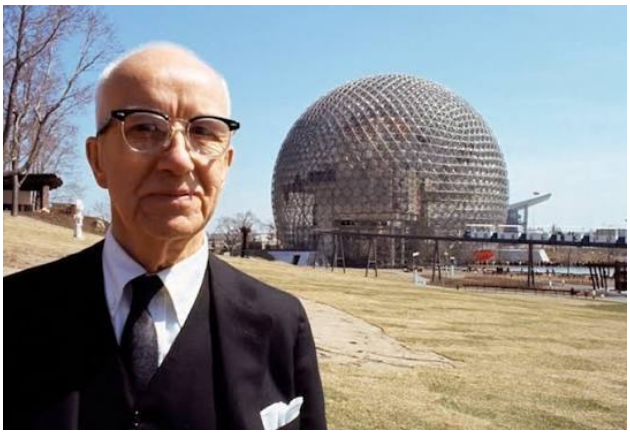
1. 基于拓扑的建模：表示进程、资源和其他系统元素之间的关系和连接。

2. 资源场抽象：探索一种模型，其中资源的可用性和需求以可以指导分配决策的方式表示。
3. 自适应调度： 研究任务调度如何响应
   变化的工作负载和系统条件。
4. 反馈驱动的协调： 利用对系统行为的观察来
   指导后续决策。
5. 涌现协调： 研究局部决策和互动是否能够
   产生有用的系统级行为。

这些是架构研究概念。它们的精确数学定义、
实施边界和已证明的性能必须通过
持续发展来确立。

第三章：巴克敏斯特・富勒与SOS的知识基础

3.1 巴克敏斯特・富勒是谁?



巴克敏斯特・富勒（1895–1983），广为人知的R. 巴克敏斯特・富勒，是一位美国发明家、建筑师、设计师和系统思想家。他的工作探讨了几何学、工程学、技术以及对互联系统的理解如何影响人类环境的设计。

他与测地线穹顶、
综合设计方法、
短暂化概念以及两卷本的著作*Synergetics*相关。

富勒不仅对设计单个物体感兴趣。他经常考虑设计决策如何影响更大的系统，包括材料、能源和技术资源的使用。

巴克敏斯特・富勒研究所将他的设计科学方法描述为发现和应用通用原则以解决实际问题的努力。
https://www.bfi.org/about-fuller/big-ideas/design-science/

3.2 什么是协同学?

富勒的*协同学*发展了一个几何和概念框架，用于探索关系、结构和系统的行为。

这项工作将几何学视为不仅仅是绘制形状的方式。它将空间关系和结构组织视为理解系统如何排列的重要主题。

富勒的思想包括对四面体和其他几何关系、结构效率以及组件如何参与更大配置的调查。

他的工作内容广泛，不应简化为一个口号或一组几何形状。它代表了更广泛的尝试，旨在考察系统的结构和关系如何影响该系统的功能。

巴克敏斯特・富勒研究所提供了对他工作的介绍，并链接到已出版的卷册。https://www.bfi.org/about-fuller/big-ideas/synergetics/

3.3 什么是短暂化?

与富勒相关的另一个重要概念是短暂化，通常总结为*用更少的资源做更多的事情*。

这一原则鼓励设计师在有限的材料、能源、时间和其他资源中寻求更大的有效性能。巴克敏斯特·富勒研究所将其视为他设计科学哲学的核心主题。https://www.bfi.org/about-fuller/big-ideas/design-science/

例如，想象两台执行相同有用任务的机器。如果一台机器在不妥协所需结果的情况下使用更少的能量完成任务，它可能提供更具资源效率的设计。

这一原则不仅仅是以任何代价减少资源消耗。一个有意义的比较必须考虑系统完成了什么，它完成得有多好，以及所需的资源。

3.4 这些思想如何激励SOS

富勒与SOS之间的联系是一种智力启发，而不是声称富勒设计了一个操作系统或提供了现成的计算机架构。

这项研究基于三个广泛的主题。

首先，关系很重要。富勒的工作邀请人们考虑组件在更大结构中的相互作用。SOS将这个问题应用于计算任务、资源和系统条件之间的关系。

其次，结构影响性能。在工程中，组件的排列可以影响强度、稳定性和材料需求。SOS研究替代计算关系的表示是否能支持有用的调度和资源管理决策。

第三，资源效率应当被审视。短暂化激发了一个问题，即一个系统是否可以用更少的处理器时间、能源、内存压力或协调开销来交付其所需的结果。

这些连接将广泛的设计理念转化为可以通过计算机科学进行研究的问题。

3.5 从哲学启发到可测量的研究

科学工作的一个重要原则是，仅有灵感并不能确立一个结果。

几何模型并不自动比传统算法更快。去中心化的安排并不自动比中心化的更高效。受富勒启发的设计并不继承他工程成就的已证明特性。

对于SOS，相关问题必须直接进行测试：

- 所提议的模型是否帮助调度器做出更好的决策？
- 它是否改善了特定工作负载的响应能力？
- 它是否减少了能耗或资源争用？
- 该模型引入了什么计算开销？
- 随着进程数量的增加，它是否仍然可靠？

这些问题提供了一条从概念启发到基于证据的评估的路径。

第4章：为什么探索替代操作系统架构？

4.1 竞争需求的挑战

计算机通常同时运行许多活动。有些需要立即响应，而其他的可以等待。有些占用大量内存，而其他的则受限于处理器速度或网络访问。

系统必须平衡这些需求。

改善一个指标的调度策略可能会恶化另一个指标。给一个任务分配更多的处理器时间可能会提高其完成时间，但会延迟其他任务。
减少能耗可能会影响性能。收集更多关于工作负载的信息可能会改善决策，但会引入额外开销。

对于每个工作负载和每个目标，没有普遍最优的策略。

这也是替代调度模型仍然值得研究的原因之一。

4.2 超越孤立的测量

传统的调度决策可能会使用任务优先级、执行时间、准备状态或资源可用性等因素。现代操作系统还可以使用复杂的依赖信息、反馈和特定于工作负载的优化。

SOS调查是否任务和资源之间关系的明确模型
可以提供额外的视角。

例如，两个任务可能会竞争同一资源。另一个任务可能依赖于其中一个任务的输出。调度决策不仅会影响立即选择的任务，还会影响相关任务的进展。

基于拓扑的表示可以使这些关系更容易表达和分析。

然而，只有在表示更多关系能够显著改善决策，以至于值得增加的计算和工程成本时，这种表示才是有用的。

4.3 研究机会

SOS旨在研究一种设计方法，将系统组织、关系和适应性视为核心架构问题
。

其潜在贡献可能包括：

- 表示任务和资源关系的新方法。
- 基于这些关系的替代调度算法。
- 表达资源需求和可用性的模型。
- 响应变化的系统条件的反馈机制。
- 评估系统整体权衡的实验方法。

目标不是假设现有的操作系统应该被抛弃。
而是确定特定的SOS概念是否能够产生可测量的好处
以及这些概念可能在哪里有用。

第五章：SOS与传统操作系统的区别

5.1 设计重点的比较

传统操作系统并不是以相同的方式构建的。Linux、Windows、macOS、
实时操作系统和嵌入式系统使用不同的架构和
调度策略。

许多操作系统已经支持复杂的资源管理、自适应行为、
依赖关系跟踪和并行执行。

因此，下面的比较关注的是SOS提出的研究重点，
而不是声称每个传统操作系统都缺乏这些能力。

设计问题	常见的操作系统方法	SOS研究重点
任务是如何被表示的?	进程、线程和其他执行抽象，取决于系统。	探索显式的关系感知任务和工作负载表示。
关系是如何被处理的?	通过依赖、同步、优先级和资源所有权等机制。	研究基于拓扑的模型以表达系统元素之间的关系。
资源是如何被分配的?	通过既定的分配政策、调度器、内存管理器和子系统特定机制。	探索资源域抽象及其在协调决策中的可能角色。
调度是如何适应的?	现有系统使用多种调度策略和反馈机制。	研究受变化关系和资源条件影响的调度策略。
如何评估效率?	取决于系统目标、工作负载和测量性能。	测试所提议的架构在定义条件下是否提高了选定的效率指标。

5.2 SOS是否更高效?

这是一个重要的问题，但需要仔细回答。

SOS旨在探索潜在的更具适应性和资源效率的计算。它尚未证明其普遍比现有操作系统更高效。

对优越效率的可信声明需要在合适的基准下进行可重复的测量。

例如，如果声称SOS调度器能提高任务完成时间，则应在相同的工作负载、硬件假设和测量程序下与传统调度算法进行测试。

结果应报告不仅是改进，还有缺点、开销以及替代方案表现不佳的情况。

5.3 效率的含义

效率不是单一的测量标准。

一个系统可能使用更少的能量，但完成任务的时间更长。另一个系统可能快速完成任务，但消耗更多的内存。调度程序可能提高平均性能，同时让最慢的任务等待太久。

因此，有效的评估应考虑多个指标。

指标	**它告诉我们的信息**
任务完成时间	任务完成所需的时间。
吞吐量	系统在一段时间内完成的工作量。
CPU 利用率	处理器的容量有多少被使用。
内存使用情况	系统消耗了多少工作内存。
每个任务的能耗	完成一定工作量所需的能量，当可测量时。
调度开销	用于做出调度决策的计算量。
响应时间变化	系统的响应时间有多一致。
公平	任务是否获得适当的 资源份额。
可扩展性	随着工作负载大小或复杂性增加，行为如何变化。

这些措施使得在计算环境中研究短暂化原理成为可能。

问题变得可测量：一个系统能否用更少的资源完成相同的工作，或在不接受不可接受的权衡的情况下，用相同的资源完成更多有用的工作？

第六章：SOS的提议架构

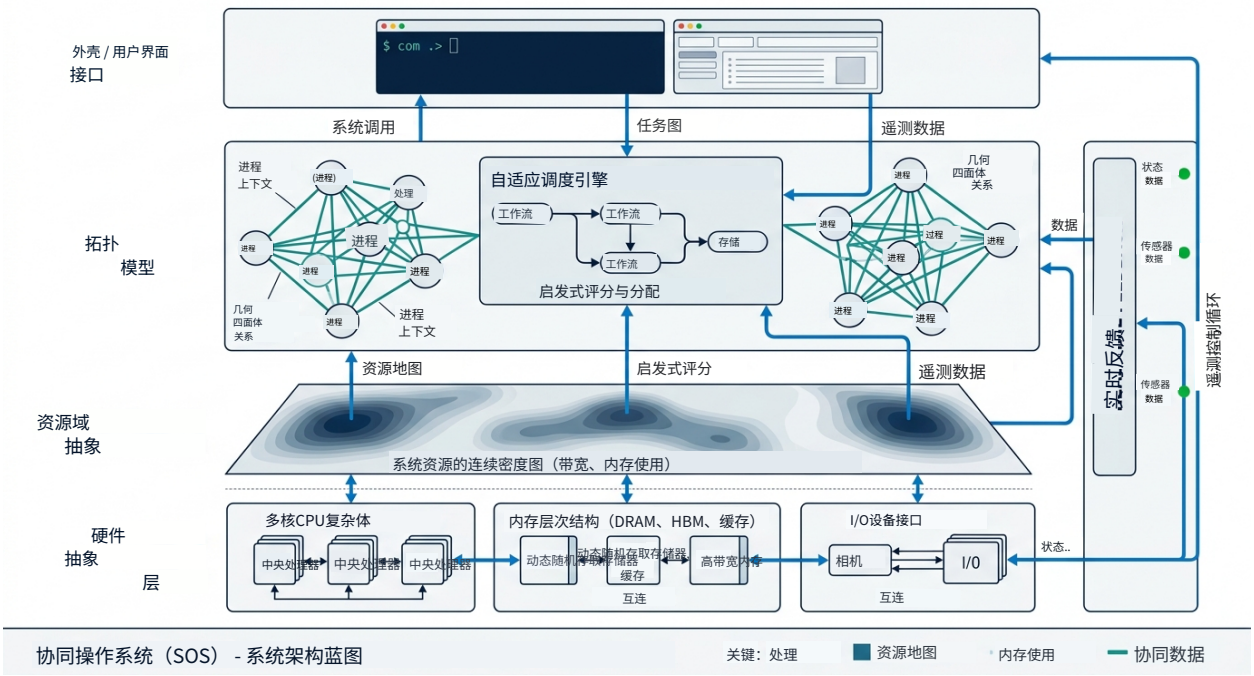
6.1 概述

SOS研究架构可以通过六个概念组件来描述：

1. 过程和工作负载模型。
2. 拓扑表示。
3. 资源域抽象。
4. 自适应调度器。
5. 反馈与评估。
6. 用户界面和系统外壳。

这些组件描述了预期架构的概念层面。它们不应被解读为每个组件已经作为一个工作操作系统子系统实现的证明。

6.2 架构图



以下图示说明了建议的信息和决策流。

流程图 TD
A[应用与工作负载]
  B[过程与工作负载模型]
  C[拓扑表示]
  D[资源域抽象]
  E[自适应调度策略]
  F[执行与资源分配]

G[监控与反馈]
  H[用户界面与研究工具]

```
    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G
    G --> D
    G --> E
```

H -. 观察并配置 .-> B
  H -. 可视化 .-> C
H -. 可视化 .-> G
图1. 概念性SOS架构。该图展示了一个研究模型，而不是一个经过验证的完整操作系统的实现图。

6.3 组件一：进程和工作负载模型

进程是一个正在运行的程序，而工作负载是系统必须处理的任务和需求的集合。

所提出的工作负载模型将描述任务的相关属性，例如：

- 它们所需的资源。
- 它们的优先级或时间要求。
- 它们对其他任务的依赖关系。
- 它们当前的执行状态。
- 随着条件变化，它们的资源需求。

考虑一个视频编辑器。导入素材、生成预览、处理特效和导出完成的视频可能涉及不同优先级和资源需求的不同任务。

工作负载模型可以帮助明确地表示这些差异。

存储的确切信息及其收集方式将取决于
实现。

6.4 组件二：拓扑表示

拓扑涉及关系和连接性。在SOS中，基于拓扑的模型
可以表示任务、资源和依赖关系是如何连接的。

一种实际的表示方式是图。

图包含节点，表示实体，以及边，表示
它们之间的关系。

例如，一个节点可以表示一个任务，而一条边则表示一个任务
依赖于另一个任务。

流程图 LR
  A[读取数据]
  B[处理数据]
  C[生成结果]
  D[保存结果]

```
    A --> B
    B --> C
    C --> D
```

图2. 一个简单的任务依赖图。

在这个例子中，处理不能开始，直到所需的数据可用，并且
结果不能保存，直到它被生成。

图表使依赖结构可见。

SOS可以研究更丰富的图表或其他拓扑表示是否能够
帮助调度决策考虑这些关系。

图表并不自动产生更好的性能。它的有用性取决于
它所表示的信息和解释它的算法。

6.5 组件三：资源领域抽象

资源领域抽象是一种提议的方式，用于表示系统中资源的可用性、需求和分布。

想象一个共享工作空间，几个人需要桌子、工具和电力。
工作空间管理者必须了解资源的可用位置、哪些需求是紧急的，以及竞争在哪里
发展。

资源领域模型旨在提供一种结构化的方式来表达计算机系统中可比较的
条件。

感兴趣的资源可能包括处理器时间、内存容量、存储访问或其他可测量的系统资源。

例如，一个模型可能描述：

- 当前可用的处理器容量。
- 活跃任务的内存需求。
- 哪些任务正在争夺共享资源。
- 资源压力是增加还是减少。
- 在系统的约束下可用的分配选项。

资源字段的数学含义必须在
实现中精确定义。除非模型
明确建立有意义的数学对应关系，否则不应与物理力场混淆。

6.6 组件四：自适应调度器

调度器负责决定哪个合格任务应该运行以及何时运行。

自适应调度器会调查调度决策是否应根据观察到的条件进行改变。

例如，当截止日期临近时，系统可能会更加关注一个时间敏感的任务，同时允许后台任务使用其他可用资源。

许多现有的操作系统已经实现了复杂的调度策略。
SOS 因此需要确定其自身的调度模型在这些既定方法之外的贡献。

一个有用的研究过程将是：

1. 定义调度问题。
2. 指定拟议的决策规则。
3. 在受控模拟中实施这些规则。
4. 将结果与基线算法进行比较。
5. 识别改进和权衡。
6. 完善模型并重复实验。

6.7 组件五：反馈与评估

反馈机制观察系统行为，并提供可以为后续决策提供信息的信息。

对于SOS，相关观察可能包括任务完成时间、资源
利用率、队列长度、内存压力和调度开销。

模型可以利用这些信息来评估其决策是否产生了预期的结果。

反馈必须经过仔细设计。收集测量数据会消耗资源，
对临时变化反应过于激烈可能会导致不稳定的行为。

因此，研究应调查反馈是否不仅改善决策，
还要评估其开销和稳定性特征是否可接受。

6.8 组件六：用户界面和系统外壳

系统外壳或图形界面提供了人们与系统交互的方式。

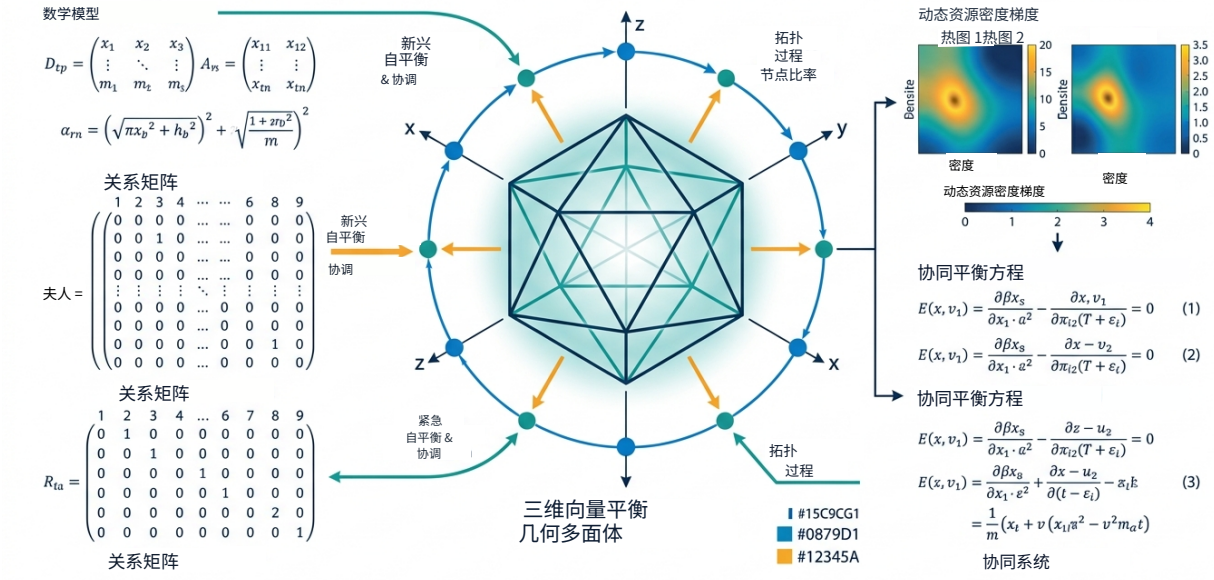
对于SOS，建议的接口可以帮助开发者和研究人员观察：

- 活动任务及其关系。
- 资源需求和可用性。
- 调度决策。
- 工作负载条件的变化。
- 实验结果和性能指标。

该接口主要是使系统行为可观察和可管理的一种方式。
一个视觉上吸引人的仪表板并不能单独证明其背后的架构是高效或可操作的。

第七章：示例设计模型

协同操作系统（SOS）的设计模型



7.1 从孤立任务到连接任务

一个传统的任务列表可能显示几个正在运行的程序：

- 浏览器。
- 视频编辑器。
- 文件传输。

- 数据处理程序。

一个关系感知的表示可以额外显示哪些任务依赖于
其他任务，哪些任务竞争资源，以及哪些任务可以独立进行。

例如，一个数据处理程序可能需要在分析下载的文件之前完成文件传输。

理解这种依赖关系可能有助于避免安排那些尚无法进行的工作。

重要的研究问题是，这种额外的表示是否在考虑维护成本后，能够比更简单的模型
产生更好的决策。

7.2 提议的调度周期

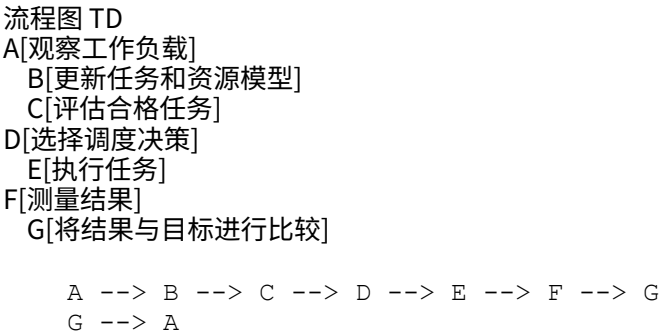


图3. 一个基于反馈的调度周期示意图。

该图展示了一般的研究工作流程，而不是声称每个步骤都已在SOS原型中实现。

7.3 设计原则：关系应支持决策

提议的架构不应仅仅为了复杂性而收集关系。

关系模型只有在帮助系统做出有用的决策时才有价值，而这些决策在其他情况下可能会
困难、昂贵或不够准确。

例如，如果一个简单的优先队列在特定工作负载下持续产生更好的结果，那么对
于该工作负载，可能不需要更复杂的拓扑模型。

一个可信的SOS实现必须愿意发现其方法的有效性以及在何种情况下更简单的技术更为
可取。

第8章：研究、模拟与原型设计

8.1 为什么要从模拟开始?

构建一个完整的操作系统是一项巨大的任务。它涉及硬件初始化、内存管理、进程隔离、设备驱动、安全、存储、调度以及许多其他组件。

因此，架构思想应在可管理的阶段中进行研究。

模拟提供了一种在不首先实现完整操作系统的情况下测试选定算法的方法。

调度器模拟可以模拟任务、资源需求、调度规则和任务完成情况。研究人员可以在受控条件下比较不同的策略。

对于SOS，一个基于Rust的调度器模拟为探索调度概念和收集证据提供了一个实际的研究方向。

调度器模拟与在可引导操作系统中运行的内核调度器并不是同一回事。模拟结果仅建立了所建模实验所支持的内容。

调度器模拟的源代码：https://github.com/synergetic-os/Simulation/blob/main/synergetic_scheduler_rust_simulation.rs

8.2 为什么选择 Rust?

Rust 是一种编程语言，强调内存安全、
性能以及对重要编程行为的明确控制。

它用于系统编程，并且是实验性能敏感调度模型的合理候选者。

使用 Rust 并不会自动使算法更快或证明系统是安全的。设计、实现、测试和验证的质量仍然至关重要。

8.3 提议的实验方法

可重复的 SOS 评估可以遵循以下程序：

1. 定义问题。 指定工作负载和资源管理问题。
2. 选择基准算法。 选择适合比较的已建立调度方法。

3. 定义 SOS 模型。 记录提议的拓扑、资源表示和决策规则。

4. 创建受控工作负载。 包括 CPU 绑定任务、资源争用、
   依赖关系以及相关的不同任务优先级。
5. 进行重复实验。使用可比较的配置和足够的
   重复次数。
6. 测量性能。 记录完成时间、吞吐量、调度开销、资源使用和其他相关指标
   。
7. 发布方法论。 提供假设、源代码和结果以供审查。

8. 调查局限性。 报告SOS表现较差、引入开销或无法扩展的情况。

8.4 公平比较的示例

假设一个提议的SOS调度算法与传统调度策略进行比较。

公平的实验应使用相同的任务集、硬件或仿真假设、资源限制和停止条件。

实验可能测量：

- 平均任务完成时间。
- 最坏情况或高百分位完成时间。
- 总调度开销。
- 吞吐量。
- 调度模型消耗的内存。

结果应以数字形式报告，实验条件应清晰记录。

在测量结果支持之前，不应声称有性能提升。

第9章：设计与用户体验

9.1 为什么要可视化系统?

当复杂系统的内部关系仅通过日志或原始数值输出可见时，理解它们可能会很困难。

可视化界面可以帮助研究人员看到任务之间的依赖关系、资源的压力点，以及调度决策如何影响系统行为。

对于SOS，有两个概念性接口方向特别相关：

- 桌面或系统外壳概念： 与系统交互和访问工具的通用环境。
- 拓扑和资源监控概念： 任务关系、资源状况和调度活动的可视化表示。

这些是设计概念。它们在文档中的存在不应被解读为接口与功能正常的SOS内核相连接的证据。

9.2 建议的概念接口图像

以下项目图像可作为准备文档时的视觉参考：

拓扑和资源监控概念

桌面接口概念

项目标志

这些插图传达了项目的设计方向。除非它们准确描绘了已实施和测试的接口，否则应标记为*概念设计*。

第10章：项目的创始人和起源

10.1 所罗门・雅威・阿德克洛

协同操作系统由所罗门・雅威・阿德克洛创立，他是一位来自加纳的软件开发者，他在SOS上的工作探讨了系统思维、资源管理和计算架构之间的关系。

该项目源于他对巴克敏斯特・富勒的协同学的兴趣，以及将更广泛的设计原则应用于操作系统研究的可能性。

他的目标是研究一种关系感知、以拓扑为导向的方法是否可以为建模和协调计算工作提供新的方式。

创始人简介：https://www.f6s.com/solomon-yaw-adeklo

10.2 从一个想法到一个研究项目

操作系统概念的开发需要的不仅仅是一个初步想法。它涉及到解释问题、定义架构、开发数学模型、实现算法、测试假设和记录结果。

SOS通过研究文档、模拟、原型开发和协作来遵循这一过程。

该项目还使用AI辅助工具，包括ChatGPT，来支持诸如
探索想法、完善文档、考虑架构和协助
编码等任务。这种帮助可以加速部分工作，但并不能替代
技术专长、测试、代码审查或独立验证。

10.3 开放协作的重要性

操作系统开发跨越多个学科。进展可能需要在Rust、内核开发、调度理论、图算
法、资源管理、测试、基准测试和技术写作方面有经验的人的贡献。

一个开源研究项目可以为这些贡献提供一个讨论、测试和改进的地方。

SOS欢迎建设性的审查。对其假设、实施和实验结果的质疑是有价值的，因为它们有助于区分有前景的概念与尚未确立的主张。

第11章：当前范围和发展路线图

11.1 为什么项目状态必须明确

一个研究项目可能包含几种不同类型的工作：已发布的概念、模拟代码、早期原型
框架、界面设计和未来的实施目标。

这些是不可互换的。

已发布的架构并不能证明其组件已被实现。模拟并不能证明操作系统可以启动。原型并不能证明生产就绪。

因此，SOS项目应分别描述每个组件的状态。

11.2 开发阶段

阶段	主要目标	所需证据
概念研究	定义问题， 原则，以及 架构。	清晰的文档， 定义，以及研究 问题。
调度器模拟		

阶段	主要目标	所需证据
	在受控条件下测试调度模型。	可重复的代码，工作负载，指标，以及结果。
原型开发	实现选定的组件和接口。	可运行的代码，测试，以及文档化的限制。
比较评估	将该方法与合适的基准进行比较。	重复实验和已发布的分析。
更深入的系统集成	调查与操作系统和硬件机制的集成。	工作组件、集成测试和安全分析。
长期的操作系统研究	探索该架构是否能够支持更完整的系统。	可证明的启动、执行、资源管理和可靠性能力，适合所声称的范围。

该表是一个路线图框架。各个里程碑的状态应从实际仓库和验证的测试结果中更新。

11.3 成功的表现形式

成功不应仅仅通过新操作系统名称的存在或视觉上独特的架构来定义。

有意义的研究贡献可以是一个在明确界定的工作负载类别中表现更好的调度算法，一个使重要关系更易于推理的资源模型，或是一个可重复的有用权衡的演示。

一个结果显示所提议的方法并未改善特定工作负载也可能是有价值的。这有助于建立模型的边界并指导未来的工作。

第12章：挑战与开放研究问题

几个问题需要仔细调查。

计算开销：维护拓扑和资源模型的成本是否超过其带来的好处？

可扩展性：随着任务和关系数量的增加，所提议的表示能否保持实用？

稳定性：基于反馈的决策能否避免振荡、饥饿和不可预测的行为？

公平性：调度器能否在不系统性地使某些工作负载处于不利地位的情况下平衡竞争任务？

安全性：架构如何在应用程序之间保持隔离、权限和保护？

可靠性： 当测量不完整、资源需求意外变化或组件故障时，系统应如何表现？

整合： 提议的模型可能如何与现有硬件、驱动程序、内存管理和内核机制相互作用？

证据： 在什么实验条件下，SOS 方法提供了可测量的优势，而在什么情况下则没有？

这些问题应被视为研究优先事项，而不是假定的解决方案。

第13章：SOS与更广泛的计算社区的关系

SOS 最好被理解为在操作系统架构和资源管理的更广泛领域内的探索。

它不需要证明每个现有系统在根本上都是有缺陷的，以证明其研究问题的合理性。

现有的操作系统体现了数十年的工程和研究。它们的调度算法、安全机制和资源管理器提供了重要的基准，以便对新想法进行评估。

SOS 可以通过清晰定义其模型、实施选定的想法、进行公平比较以及发布正面和负面的发现来有意义地贡献。

该项目从富勒（Fuller）获得的灵感增添了独特的设计视角，但其技术贡献最终必须通过计算机科学和工程证据进行评估。

第14章：合作与参与

该项目欢迎对探索替代计算方法感兴趣的人员贡献。

潜在的贡献领域包括：

- Rust 编程和代码审查。
- 操作系统架构。
- 调度算法和图论。
- 数学建模和优化。
- 资源管理和性能测量。
- 测试、基准测试和可重复性。
- 文档、研究和文献综述。
- 界面设计和系统可视化。

贡献者可以通过审查假设、改善定义、构建小型
实验组件、测试算法或使项目更易于
理解来提供帮助。

要表达兴趣，请联系：

邮箱：synergeticsoperatingsystem@gmail.com

官方网站：https://synergetic-os.github.io/

GitHub组织：https://github.com/synergetic-os

创始人简介：https://www.f6s.com/solomon-yaw-adeklo

该项目应保持明确的贡献指南、适当的许可、
透明的归属和记录的审查及接受工作的程序。

第15章：结论 — 探索一种不同的计算思维方式

协同操作系统以一个问题开始：不同的方式来表示任务、资源和系统组件之间的关系，是否能导致有用的新计算方法？

巴克敏斯特·富勒的协同学为研究关系和系统结构提供了智力启发。他的短暂化概念鼓励设计师探讨如何用更少的资源实现有用的结果。

SOS将这些广泛的思想转化为一个研究方向，涉及基于拓扑的建模、资源场抽象、自适应调度和反馈驱动的协调。

机会在于不假设所提议的架构已经优越。它在于构建模型、开发算法、将其与既定方法进行测试，并发布证据以便他人评估结果。

该项目的长期价值将取决于其研究的质量、实施的清晰度、实验的可重复性以及更广泛计算社区的贡献。

当操作系统设计的新方法的思想能够被审视、实施、测试、挑战和改进时，它才变得有意义。

SOS的雄心是探索一种更明确关注关系的计算方法是否能帮助系统更智能地使用其资源。
研究仍在进行中，其结论必须通过
证据来获得。

参考文献与进一步阅读

1. **Synergetics 操作系统 — 官方网站**
   https://synergetic-os.github.io/
2. **Synergetics 操作系统 — GitHub 组织**
   https://github.com/synergetic-os
3. Solomon Yaw Adeklo — F6S 个人资料
   https://www.f6s.com/solomon-yaw-adeklo
4. Buckminster Fuller 研究所 — Synergetics
   https://www.bfi.org/about-fuller/big-ideas/synergetics/
5. Buckminster Fuller 研究所 — 设计科学
   https://www.bfi.org/about-fuller/big-ideas/design-science/
6. Fuller 的 Synergetics — 在线参考
   http://www.rwgrayprojects.com/synergetics/intro/explicit.html
7. SOS 研究文章 — DEV Community
   https://dev.to/synergeticos/rethinking-the-operating-system-why-we-need-to-explore-a-synergetic-approach-to-computing-507g
8. **SOS 研究文章 — Medium**
   https://sadeklo.medium.com/rethinking-the-operating-system-why-we-need-to-explore-a-synergetic-approach-to-computing-22320491b17c

---

文档状态：研究概述和概念架构。实施
声明、基准结果和里程碑状态应在验证的项目
证据可用时进行更新。

推荐的下一步

上述文本提供了第一份全面的草稿。要将其转变为正式的研究文档，下一阶段应添加实际的SOS架构和设计图，插入生成的封面图像，并直接根据项目代码库验证当前的实施状态。该草稿中的图表明确是概念性的，因此读者不会将其误认为是完整操作系统的证明。